KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

DEPARTMENT OF COMPUTER TECHNOLOGY - PG

M.Sc (Software Systems)

Research Project

FirmGraph-LLM: A Knowledge Graph-Guided Framework for Cross-Firmware Vulnerability Analysis

Presented By:

Dharshan P

23ISR012

Guided By:

Mr C Rukumani Khandhan,

Assistant Professor,

Department of CT-PG,

Kongu Engineering College,

Perundurai.

Estd : 1984

18/7/2026

Problem Statement

- The increasing number of IoT devices has expanded the firmware attack surface, making firmware security a critical challenge.
- Existing firmware analysis techniques rely on manual reverse engineering and architecture-specific tools, limiting scalability and efficiency.
- Current LLM-based binary analysis frameworks analyze firmware independently without reusing knowledge from previous analyses.
- There is a need for an intelligent, explainable framework that combines Knowledge Graphs and GraphRAG to enable accurate cross-firmware vulnerability discovery.

Introduction

- The rapid growth of IoT and embedded devices has increased the need for secure firmware to protect connected systems from cyber threats.
- Traditional firmware analysis relies on manual reverse engineering, making vulnerability discovery time-consuming and difficult to scale.
- Large Language Models (LLMs) have improved binary code analysis but often analyze firmware independently without retaining prior knowledge.
- Existing approaches lack cross-firmware knowledge reuse and structured reasoning, limiting accurate and explainable vulnerability analysis.
- FirmGraph-LLM integrates Knowledge Graphs, GraphRAG, and LLMs to enable reusable, explainable, and intelligent firmware vulnerability analysis.e accurate, evidence-based, and explainable medical recommendations.

Objective

Develop a Knowledge Graph-guided Large Language Model (LLM) framework for cross-firmware vulnerability discovery that combines firmware reverse engineering, structured knowledge representation, and Retrieval-Augmented Generation (GraphRAG) to provide accurate, explainable, and reusable vulnerability analysis across diverse IoT firmware families.

Literature Survey

Base Paper: *LLM-Powered Static Binary Taint Analysis (LATTE)* proposes an LLM-assisted framework for static binary taint analysis, improving vulnerability detection through semantic reasoning over binary code. However, the framework analyzes each binary independently and lacks cross-firmware knowledge reuse, Knowledge Graph integration, and GraphRAG-based explainable reasoning, which motivates the proposed FirmGraph-LLM framework.

Table of surveyed papers

S.No	Title of the Paper	Journal	Methodology Proposed	Research Gap Addressed
1	LLM-Powered Static Binary Taint Analysis (LATTE)	ACM TOSEM (2025)	LLM-guided prompt sequences + binary taint analysis	Analyzes each binary independently; lacks cross-firmware KG/GraphRAG.
2	Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG	ACM TOSEM / arXiv (2024)	CVE knowledge base + RAG + LLM reasoning	Source-code focused; not designed for firmware binaries.
3	VulBinLLM: LLM-powered Vulnerability Detection for Stripped Binaries	arXiv (2025)	Optimized decompilation + CoT + memory agent	No reusable firmware knowledge across analyses.
4	GRAG: Graph Retrieval-Augmented Generation	arXiv (2024)	Graph Retrieval + LLM	No binary analysis or firmware application.
5	FITS: Inferring Intermediate Taint Sources for Effective Vulnerability Analysis	ASPLOS (2024)	Intermediate taint-source inference + behavioral features	Static analysis only; no persistent cross-firmware learning.
6	FirmAgent: Leveraging Fuzzing to Assist LLM Agents with IoT Firmware Vulnerability Discovery	NDSS (2026)	Fuzzing feedback + LLM agents + PoC refinement	Runtime-heavy; lacks structured GraphRAG memory.

KONGU ENGINEERING COLLEGE

Table of surveyed papers

S.No	Title of the Paper	Journal	Methodology Proposed	Research Gap Addressed
7	FIRMHIVE: LLMs as Firmware Experts: A Runtime-Grown Tree-of-Agents Framework	arXiv (2025)	Recursive Tree-of-Agents + cross-file firmware reasoning	High compute cost; session memory, not reusable KG.
8	FirmCure: Towards Autonomous and Adaptive Rehosting of Linux-Based Firmware	arXiv (2026)	LLM-driven rehosting + reflective configuration synthesis	Focuses on rehosting, not vulnerability knowledge reuse.
9	From Local to Global: A GraphRAG Approach to Query-Focused Summarization	Microsoft / arXiv (2024)	Entity graph + community summaries + GraphRAG	Text-corpus oriented; not adapted for firmware binaries.
10	CyKG-RAG: Towards Knowledge-Graph Enhanced RAG for Cybersecurity	CEUR / ISWC Workshop (2024)	Cybersecurity KG + symbolic/vector retrieval + LLM	General cyber QA; no firmware-level function graph.
11	GraphReader: Building Graph-based Agents to Enhance Long-Context Abilities of LLMs	arXiv (2024)	Knowledge Graph + LLM Agent	General graph reasoning; not firmware security.
12	Detecting Vulnerability on IoT Device Firmware: A Survey	IEEE Access (2024)	Survey of Firmware Analysis Techniques	Identifies challenges but proposes no automated framework.

Novelty

- Enables cross-firmware knowledge reuse through GraphRAG, reducing redundant analysis and improving vulnerability discovery.
- Integrates a Firmware Knowledge Graph with LLMs to provide structured, context-aware vulnerability analysis across diverse firmware.
- Generates explainable vulnerability reports with reasoning paths, supporting evidence, and confidence scores for analyst trust.
- Continuously updates the Knowledge Graph with newly discovered vulnerability patterns, enabling scalable and adaptive firmware security analysis.

KONGU ENGINEERING COLLEGE

Dataset Description

Description:

The proposed system uses public firmware images, vulnerability databases and extracted binary artefacts to build a firmware knowledge graph and evaluate cross-firmware vulnerability reasoning.

Category	Dataset / Source	Purpose
Firmware Images	OpenWrt / vendor firmware	Firmware extraction and analysis
Firmware Emulation	FirmAE / Firmadyne samples	Testing and baseline comparison
Vulnerability Data	NVD CVE Database	CVE mapping and validation
Weakness Taxonomy	MITRE CWE	Classifying vulnerability types
Binary Artefacts	Ghidra-extracted functions	KG nodes and embeddings

Estd : 1984

Dataset Description

Data Collection and Data Preprocessing

Data Collection

- Collect firmware images from OpenWrt, vendor repositories and public firmware datasets.
- Gather CVE/CWE metadata from NVD and MITRE.
- Extract binaries, libraries, strings and configuration files using Binwalk.

Data Preprocessing

- Remove duplicate or corrupted firmware samples.
- Detect architecture, file type and executable format.
- Extract functions, CFG, APIs and strings using Ghidra.
- Generate embeddings and store graph/vector indexes for GraphRAG.

Proposed Methodology

System Workflow

Firmware Acquisition – Collect IoT firmware images from public repositories or vendor sources.

Firmware Extraction – Unpack file systems, binaries, libraries and configuration files.

Reverse Engineering – Recover functions, call graphs, strings and security artefacts using Ghidra.

Knowledge Graph Construction – Link firmware, functions, APIs, CVEs, CWEs and exploit primitives in Neo4j.

GraphRAG Retrieval – Retrieve similar vulnerability patterns and evidence from the knowledge graph.

LLM Vulnerability Analysis – Rank vulnerable functions using retrieved graph evidence.

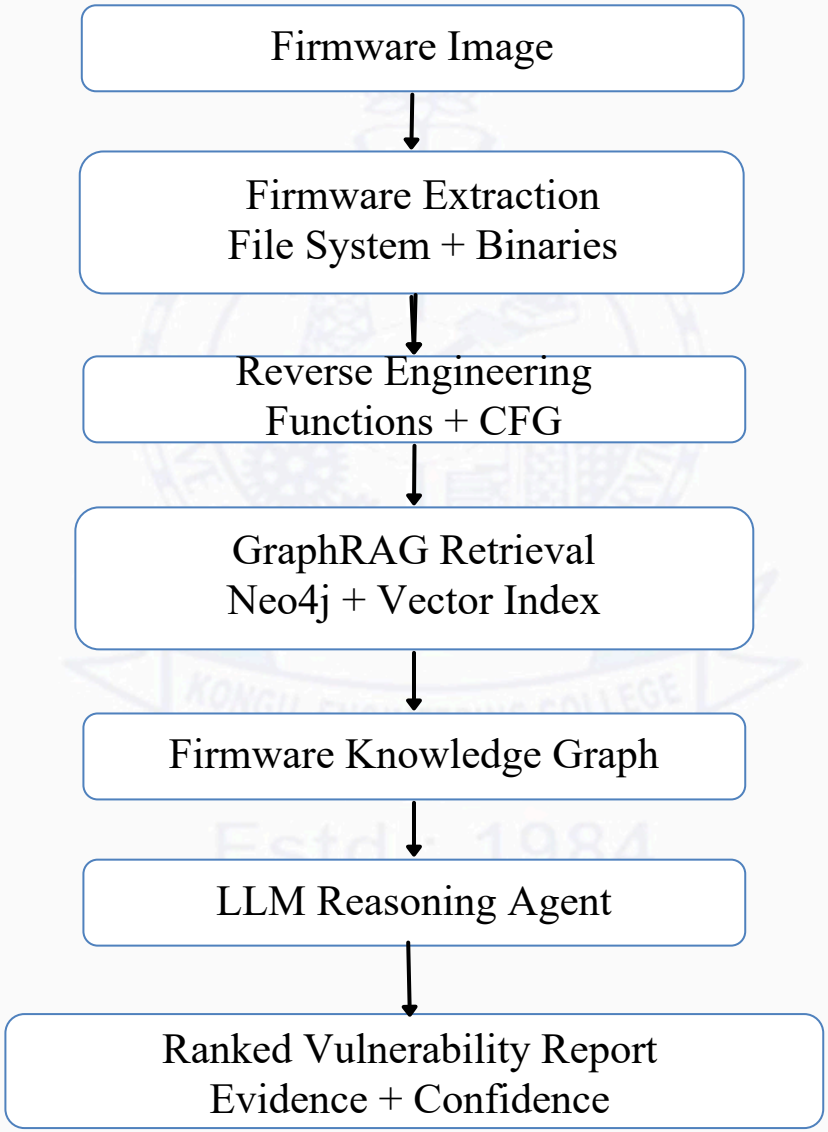
Report Generation – Produce explainable findings with confidence score and supporting evidence.

KONGU ENGINEERING COLLEGE

Proposed Methodology

Component	Method / Technology
Firmware Extraction	Binwalk / Firmware-Mod-Kit
Reverse Engineering	Ghidra / angr
Feature Extraction	Functions, APIs, strings, CFG, call graph
Knowledge Graph	Neo4j + CWE/CVE relationships
Graph Retrieval	GraphRAG + similarity search
Embedding Model	CodeBERT / UniXcoder / Node2Vec
LLM Reasoning	Gemini / GPT / Qwen-Coder
Final Response	Explainable report with confidence score

Proposed System Architecture



Tools and Technologies

Category	Technology
Programming Language	Python
Firmware Extraction	Binwalk, Firmware-Mod-Kit
Reverse Engineering	Ghidra, angr, Radare2
Knowledge Graph	Neo4j, NetworkX
GraphRAG Framework	LangChain / LangGraph
Large Language Model	Qwen-Coder
Embedding Model	CodeBERT / UniXcoder
Vector Database	FAISS / ChromaDB
Vulnerability Sources	NVD, MITRE CVE, CWE
AI & Deep Learning	Hugging Face, PyTorch
Backend / API	FastAPI
Deployment	Docker, Linux

KONGU ENGINEERING COLLEGE

Workflow

Phase 1- Ingestion & core scaffolding

- stablished a modular and reproducible foundation for the FirmGraph-LLM pipeline.
- Implemented SHA-256, SHA-1 and MD5 fingerprinting to give each firmware sample a deterministic identity.
- Added python-magic based file identification to correctly recognize ELF, ZIP, U-Boot and raw firmware formats.
- Added metadata persistence to maintain traceability and reproducibility across experiments.

KONGU ENGINEERING COLLEGE

Phase 2 - Ghidra CFG

- Integrated Juliet C/C++ v1.3 to provide controlled CWE-based vulnerability ground truth.
- Developed CWE-wise compilation automation to convert source test cases into consistent evaluation binaries.
- Prepared headless Ghidra analysis to extract functions, CFGs and dangerous-flow information.
- This converts raw vulnerability samples into structured binary-analysis evidence for the LLM.
- Status: In Progress — dataset staging and ground-truth mapping completed.
- Ghidra is suitable here because it provides disassembly, decompilation, graphing and scripting capabilities for automated reverse engineering.

KONGU ENGINEERING COLLEGE

Phase 3 - Neo4j Knowledge Graph & GraphRAG

- Built the Neo4j Knowledge Graph to preserve relationships between functions, APIs, CWEs and firmware samples.
- Added CWE taxonomy and known-API knowledge to provide security-specific context.
- Integrated FAISS to combine semantic similarity with graph-based retrieval.
- Implemented GraphRAGRetriever so the LLM receives connected historical evidence instead of isolated information.
- This addresses the project's statefulness gap by enabling knowledge reuse across firmware analyses.

Phase 4 — Comparative Evaluation

- Developed an automated evaluation engine to provide a reproducible comparison.
- Implemented LATTE as the stateless baseline and FirmGraph-LLM as the GraphRAG-enabled approach.
- Added TP, FP, TN, FN, Precision, Recall, F1 and Accuracy calculations.
- Added Top-K Recall (K=1,3,5,10) and MRR to evaluate vulnerability ranking.
- Automated delta comparison to determine whether persistent graph-based knowledge actually improves vulnerability discovery.

KONGU ENGINEERING COLLEGE

Key Enhancements

- Contextual Accuracy – Uses Knowledge Graph relationships and cross-firmware evidence for richer vulnerability reasoning.
- LLM Token Usage – GraphRAG retrieves only relevant nodes and relationships, reducing unnecessary context sent to the LLM.
- Hallucination – Grounds LLM responses using binary evidence, CWE knowledge and graph relationships.
- Cross-Firmware Knowledge Reuse – Preserves previously identified vulnerability patterns and reuses them when analyzing new firmware.
- Explainability – Provides traceable evidence paths such as Function → API → CWE → Vulnerability Pattern.
- Analysis Isolation – Changes independent per-binary analysis into persistent, reusable security knowledge.

KONGU ENGINEERING COLLEGE

Conclusion

- Enhances vulnerability analysis through Knowledge Graph and GraphRAG-based contextual reasoning.
- Reduces LLM token usage and hallucination by retrieving only relevant, evidence-grounded security context.
- Enables cross-firmware knowledge reuse by persistently storing vulnerability patterns, APIs, functions, and CWE relationships.
- Improves explainability and prioritization through graph-based evidence paths and combined historical/structural analysis.

KONGU ENGINEERING COLLEGE
KONGU ENGINEERING COLLEGE PERUNDURAI
Estd : 1984

Phase 4 — Comparative Evaluation

- Developed an automated evaluation engine to provide a reproducible comparison.
- Implemented LATTE as the stateless baseline and FirmGraph-LLM as the GraphRAG-enabled approach.
- Added TP, FP, TN, FN, Precision, Recall, F1 and Accuracy calculations.
- Added Top-K Recall (K=1,3,5,10) and MRR to evaluate vulnerability ranking.
- Automated delta comparison to determine whether persistent graph-based knowledge actually improves vulnerability discovery.

KONGU ENGINEERING COLLEGE

KONGU ENGINEERING COLLEGE

Thank You !!